



PHASE 1 : PYTHON FONDAMENTAUX
Parcours Data Science - Akieni Academy - Cohorte 2

PROJET PRATIQUE - S6

Digitaliser les récoltes du reseau cooperatif AgriSuds

1. Contexte du projet

AgriSuds est un reseau cooperatif qui accompagne des petits producteurs agricoles dans plusieurs pays d'Afrique centrale et de l'Ouest. Il regroupe cinq antennes locales, chacune rattachee a une zone de production :

- Owando - Congo-Brazzaville (manioc, huile de palme)
- Divo - Cote d'Ivoire (cacao, cafe)
- Bafoussam - Cameroun (maïs, arachide, cafe)
- Kaolack - Senegal (arachide, mil, riz)
- Kpalime - Togo (cacao, cafe, banane plantain)

Chaque antenne suit, pour ses producteurs affiliates, l'identite de l'agriculteur, la culture concernee, la quantite recoltee (en kg), le prix de vente au kilogramme (en FCFA) et l'anciennete de la recolte (en jours). Ces informations sont deja mises a votre disposition dans le notebook, sous forme de listes Python pretes a l'emploi : l'objectif du projet est de les exploiter, pas de les collecter.

1.1 Le besoin exprime par AgriSuds

La direction d'AgriSuds a formule une demande claire a Akieni Academy : construire, en Python, un ensemble de fonctions capables de repondre sans intervention manuelle aux questions suivantes :

- Quel est le chiffre d'affaires total genere par l'ensemble des recoltes ?
- Quelle quantite a ete recoltee pour une culture donnee (cacao, cafe, manioc...) ?
- Combien de recoltes ont ete enregistrees pour une antenne donnee ?
- Quelles sont, au total, les cultures et les antennes actives dans les donnees ?
- Quelle fourchette de prix (minimum / maximum) a ete pratiquee pour une culture ?
- Quelle est la fiche complete d'un producteur donne (antenne, cultures, quantite totale) ?
- Qui sont les producteurs les plus productifs, pour les valoriser lors des assemblees annuelles ?
- Quelles recoltes ont eu lieu recemment (ex. 30 derniers jours), pour anticiper les livraisons ?
- Quelle est la part (en %) de chaque culture dans la production totale ?

1.2 Pourquoi ce cas s'y prete particulierement

Ce cas d'usage correspond exactement aux notions deja vues en cours : les donnees de plusieurs dizaines de recoltes ne peuvent plus etre gerees avec de simples variables isolees - il faut des listes pour les stocker, des tuples pour renvoyer plusieurs resultats a la fois (une fourchette de prix, par exemple), des dictionnaires pour construire une fiche de synthese ou une repartition en pourcentage, et des sets pour obtenir instantanement la liste des cultures ou antennes sans doublon. Aucun module externe et aucune lecture de fichier ne sont necessaires : tout se joue avec les structures de donnees fondamentales de Python.

2. Fiche projet

Element	Detail
Domaine	Agriculture - cooperatives agricoles en Afrique
Structure commanditaire	Reseau cooperatif AgriSuds
Parcours	Data Science - Akieni Academy - Cohorte 2
Notions mobilisees	Variables, conditions, boucles, fonctions, listes, tuples, dictionnaires, sets
Format des donnees	7 listes Python paralleles, deja fournies dans le notebook (50 recoltes simulees)
Livrables	Notebook complete (.ipynb) + synthese redigee
Rendu	Individuel (travail prepare en groupe de 3 a 5)

3. Donnees fournies

Les donnees sont deja chargees dans le notebook sous forme de 7 listes paralleles (la recolte d'indice i est decrite par l'element d'indice i dans chacune des listes). Aucune lecture de fichier n'est necessaire : il suffit d'executer la premiere cellule du notebook.

Liste	Type des elements	Exemple	Remarque
IDS	entier	1	Identifiant unique de la recolte
AGRICULTEURS	texte	Aminata Mokoko	Prenom + nom du producteur
ANTENNES	texte	Owando	Antenne locale de rattachement
CULTURES	texte	Cacao	Une des 8 cultures suivies
QUANTITES_KG	decimal ou None	245.3	Peut etre None : donnee manquante sur le terrain (~8% des cas)
PRIX_KG_FCFA	entier	1520	Prix de vente au kilogramme, en FCFA
JOURS_ECOULE	entier	15	Anciennete de la recolte, en nombre de jours

Remarque : environ 8% des recoltes ont une quantite manquante (valeur None dans `QUANTITES_KG`), pour simuler un oubli de saisie sur le terrain. Vos fonctions doivent gerer ce cas sans planter.

4. Votre mission

Vous devez completer, dans le notebook fourni, une serie de fonctions organisees en trois niveaux de difficulte croissante, plus un bonus facultatif. Chaque fonction recoit en parametres les listes de donnees deja fournies, et renvoie un resultat precis (nombre, tuple, dictionnaire ou liste selon les cas).

Une cellule de test suit immediatement chaque fonction : completez une fonction, executez sa cellule de test tout de suite pour verifier votre resultat, puis passez a la fonction suivante. Inutile d'attendre la fin du notebook pour savoir si votre code fonctionne.

5. Detail des fonctions attendues

Niveau Debutant

5.1 `chiffre_affaires_total(quantites, prix)`

- Initialiser un total a 0.
- Parcourir toutes les recoltes a l'aide de leur indice i.
- Si la quantite de la recolte i est absente (None), passer a la recolte suivante sans rien ajouter.
- Sinon, ajouter au total le produit (quantite x prix) de cette recolte.
- Renvoyer le total.

5.2 `quantite_totale_culture(cultures, quantites, culture_cible)`

- Initialiser un total a 0.
- Parcourir toutes les recoltes a l'aide de leur indice i.
- Si la culture de la recolte i correspond a la culture recherchee ET que la quantite n'est pas absente, ajouter cette quantite au total.
- Renvoyer le total.

5.3 `nombre_recoltes_antenne(antennes, antenne_cible)`

- Initialiser un compteur a 0.
- Parcourir la liste des antennes a l'aide de l'indice i.
- Si l'antenne de la recolte i correspond a l'antenne recherchee, incrementer le compteur de 1.
- Renvoyer le compteur.

Niveau Intermediaire

5.4 `cultures_et_antennes_uniques(cultures, antennes)`

- Creer un set vide pour les cultures, et un set vide pour les antennes.
- Parcourir la liste des cultures et ajouter chaque valeur au set des cultures.
- Parcourir la liste des antennes et ajouter chaque valeur au set des antennes.
- Renvoyer les deux sets reunis dans un tuple : (cultures_uniques, antennes_uniques).

5.5 `fourchette_prix_culture(cultures, prix, culture_cible)`

- Creer une liste vide destinee a recevoir les prix de la culture recherchee.
- Parcourir toutes les recoltes ; si la culture correspond a la culture recherchee, ajouter son prix a cette liste.
- Calculer le prix minimum et le prix maximum de cette liste.
- Renvoyer ces deux valeurs sous forme d'un tuple (prix_min, prix_max).

5.6 `fiche_producteur(agriculteurs, antennes, cultures, quantites, nom_cible)`

- Preparer une variable pour l'antenne du producteur, un set vide pour ses cultures, et un total a 0 pour sa quantite.
- Parcourir toutes les recoltes ; des que le nom du producteur correspond au nom recherche : memoriser son antenne, ajouter la culture de cette recolte a son set de cultures, et ajouter la quantite au total (si elle n'est pas absente).
- Construire et renvoyer un dictionnaire avec trois cles : "antenne", "cultures" et "quantite_totale".

Niveau Avance

5.7 `top_n_producteurs(agriculteurs, quantites, n)`

- Cumuler, pour chaque producteur, sa quantite totale recoltee (une seule entree par producteur).
- Construire une liste de tuples (nom_producteur, quantite_totale).
- Trier cette liste du plus grand total au plus petit, a l'aide de la fonction sorted() et de son parametre key.
- Ne garder que les n premiers elements de la liste trie.
- Renvoyer cette liste reduite aux n meilleurs producteurs.

5.8 recoltes_recentes(jours_ecoule, quantites, prix, nb_jours)

- Parcourir toutes les recoltes a l'aide de leur indice i.
- Si l'anciennete de la recolte i (jours_ecoule[i]) est inferieure ou egale a nb_jours ET que la quantite n'est pas absente, ajouter (quantite x prix) au total.
- Renvoyer le total obtenu.

5.9 repartition_par_culture_pourcentage(cultures, quantites, cultures_uniques)

- Pour chaque culture du set fourni, calculer sa quantite totale a l'aide de quantite_totale_culture(...) et la stocker dans un dictionnaire {culture: quantite_totale}.
- Calculer la quantite totale generale, tous produits confondus, avec sum() sur les valeurs du dictionnaire.
- Pour chaque culture, calculer son pourcentage dans le total general (quantite_culture / quantite_totale_generale x 100), arrondi a 1 decimale.
- Renvoyer un dictionnaire {culture: pourcentage}.

Bonus (facultatif)

5.10 producteur_plus_actif_par_antenne(agriculteurs, antennes, quantites)

- Construire un dictionnaire intermediaire {(antenne, producteur): quantite_totale} en parcourant les recoltes (ignorer les quantites manquantes).
- Pour chaque antenne, identifier le producteur dont la quantite totale est la plus elevee.
- Renvoyer un dictionnaire {antenne: nom_du_meilleur_producteur}.

6. Visualisation

Une fois vos fonctions completees, une cellule fournie genere automatiquement 4 graphiques a partir de vos resultats : quantite totale par culture, quantite totale par antenne, repartition en pourcentage des cultures (camembert), et top 5 des producteurs (barres horizontales).

7. Livrable attendu et critères d'evaluation

Le livrable est le notebook complete (fichier .ipynb), incluant la cellule de synthese redigee en francais a la fin du document. Le travail peut etre prepare et discute en groupe (brainstorming, entraide sur les points bloquants), mais chaque apprenant code et rend son propre notebook, individuellement.

Critere	Points
Niveau Debutant (3 fonctions)	6
Niveau Intermediaire (3 fonctions)	6
Niveau Avance (3 fonctions)	6
Synthese redigee	2
Bonus (producteur_plus_actif_par_antenne)	+1 (hors bareme)
Total	20